

Phase 2 (CLO2): Binary Tree, BST & Graph Traversal

Complete simple and viva-friendly Python notes

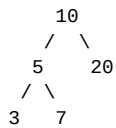
PART A — BINARY TREE

1. What is a Binary Tree?

A **Binary Tree** is a tree data structure in which each node can have **at most two children**. These children are called the **left child** and **right child**.

Important: A binary tree does **not** require the values to be sorted. The main rule is simply that every node has zero, one, or two children.

Example:



Here, 10 is the root. It has two children, 5 and 20. Node 5 also has two children, 3 and 7. Nodes 3, 7, and 20 are leaf nodes because they have no children.

2. Basic Terminology

- **Root:** The topmost node of the tree.
- **Parent:** A node that has a child.
- **Child:** A node directly connected below a parent.
- **Leaf:** A node with no children.
- **Edge:** The connection between two nodes.
- **Height:** The longest path from a node to a leaf, depending on the convention used.

3. Types of Binary Trees

- **Full Binary Tree:** Every node has either 0 or 2 children.
- **Complete Binary Tree:** All levels are full except possibly the last, and the last level is filled from left to right.
- **Perfect Binary Tree:** Every internal node has two children and all leaves are at the same level.
- **Skewed Binary Tree:** Nodes mostly continue in one direction, making the tree similar to a linked list.

4. Binary Tree Python Implementation

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Create nodes
root = Node(10)
root.left = Node(5)
root.right = Node(20)
root.left.left = Node(3)
root.left.right = Node(7)
```

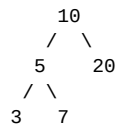
The **Node** class stores the value and two references: left and right. Initially both references are None. We then connect nodes to build the tree.

5. Binary Tree Traversals

Traversal means visiting every node in a particular order. The three common depth-first traversals are:

- **Preorder:** Root → Left → Right
- **Inorder:** Left → Root → Right
- **Postorder:** Left → Right → Root

Tree:



Preorder: 10, 5, 3, 7, 20

Inorder: 3, 5, 7, 10, 20

Postorder: 3, 7, 5, 20, 10

6. Traversal Code

```
def preorder(node):
    if node is not None:
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)

def inorder(node):
    if node is not None:
        inorder(node.left)
        print(node.data, end=" ")
        inorder(node.right)

def postorder(node):
    if node is not None:
        postorder(node.left)
        postorder(node.right)
        print(node.data, end=" ")
```

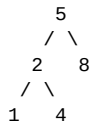
The condition **if node is not None** stops recursion when there is no node. Each traversal changes only the position of the print statement.

PART B — BINARY TREE vs BINARY SEARCH TREE

7. What is a BST?

A **Binary Search Tree (BST)** is a special type of binary tree. In addition to having at most two children, it follows an ordering rule: smaller keys go to the left and larger keys go to the right.

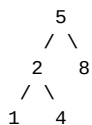
BST:



Feature	Binary Tree	Binary Search Tree
Main rule	At most 2 children	At most 2 children + ordering
Ordering	No required order	Left < Root < Right
Searching	Usually O(n)	Average O(log n) if balanced
Worst case	O(n)	O(n) if skewed
Inorder	Not necessarily sorted	Sorted keys

8. BST Example and Search

For keys 5, 2, 8, 1, and 4, the BST becomes:



To search for 4: start at 5 → 4 is smaller, so go left → reach 2 → 4 is larger, so go right → reach 4 → found.

9. BST Python Code

```
class BSTNode:
    def __init__(self, key, data=None):
        self.key = key
        self.data = data or {}
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key, data=None):
        if self.root is None:
            self.root = BSTNode(key, data)
        else:
            self._insert_recursive(self.root, key, data)

    def _insert_recursive(self, current, key, data):
        if key < current.key:
            if current.left is None:
                current.left = BSTNode(key, data)
            else:
                self._insert_recursive(current.left, key, data)
        elif key > current.key:
            if current.right is None:
                current.right = BSTNode(key, data)
            else:
                self._insert_recursive(current.right, key, data)
        else:
            current.data = data

    def search(self, key):
        return self._search_recursive(self.root, key)

    def _search_recursive(self, current, key):
```

```

    if current is None or current.key == key:
        return current
    if key < current.key:
        return self._search_recursive(current.left, key)
    return self._search_recursive(current.right, key)

def inorder(self, node=None, is_root=True):
    if is_root and node is None:
        node = self.root
    if node is None:
        return []
    return (self.inorder(node.left, False)
            + [node]
            + self.inorder(node.right, False))

```

10. BST Inorder Traversal

Inorder follows Left → Root → Right. For the BST above the result is **1, 2, 4, 5, 8**. This is sorted because the tree follows BST ordering.

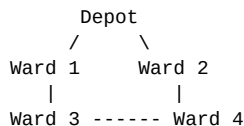
PART C — GRAPH AND ADJACENCY LIST

11. What is a Graph?

A **graph** is a collection of vertices (nodes) connected by edges. Graphs can represent locations, roads, departments, networks, and relationships.

Adjacency list: Each node is stored with the nodes directly connected to it.

```
graph = {
    "Depot": ["Ward 1", "Ward 2"],
    "Ward 1": ["Depot", "Ward 3"],
    "Ward 2": ["Depot", "Ward 4"],
    "Ward 3": ["Ward 1", "Ward 4"],
    "Ward 4": ["Ward 2", "Ward 3"]
}
```



PART D — BFS

12. Breadth-First Search (BFS)

BFS means **Breadth-First Search**. It explores the graph level by level. It normally uses a **queue**, which follows FIFO: First In, First Out.

```
def bfs(graph, start_node):
    visited = []
    queue = [start_node]

    while queue:
        current = queue.pop(0)

        if current not in visited:
            visited.append(current)

            for neighbor in graph.get(current, []):
                if neighbor not in visited and neighbor not in queue:
                    queue.append(neighbor)

    return visited
```

Starting from Depot:

1. Visit Depot and add Ward 1, Ward 2.
2. Visit Ward 1 and add Ward 3.
3. Visit Ward 2 and add Ward 4.
4. Visit Ward 3.
5. Visit Ward 4.

BFS result: Depot → Ward 1 → Ward 2 → Ward 3 → Ward 4

PART E — DFS

13. Depth-First Search (DFS)

DFS means **Depth-First Search**. It follows one branch as deeply as possible before backtracking. This implementation uses **recursion**, where the function call stack behaves like a stack.

```
def dfs(graph, start_node, visited=None):
    if visited is None:
        visited = []

    visited.append(start_node)

    for neighbor in graph.get(start_node, []):
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

return visited

Starting from Depot: Depot → Ward 1 → Ward 3 → Ward 4 → Ward 2.

PART F — COMPLETE GRAPH PROGRAM

```
graph = {
    "Depot": ["Ward 1", "Ward 2"],
    "Ward 1": ["Depot", "Ward 3"],
    "Ward 2": ["Depot", "Ward 4"],
    "Ward 3": ["Ward 1", "Ward 4"],
    "Ward 4": ["Ward 2", "Ward 3"]
}

def bfs(graph, start_node):
    visited = []
    queue = [start_node]

    while queue:
        current = queue.pop(0)
        if current not in visited:
            visited.append(current)
            for neighbor in graph.get(current, []):
                if neighbor not in visited and neighbor not in queue:
                    queue.append(neighbor)
    return visited

def dfs(graph, start_node, visited=None):
    if visited is None:
        visited = []
    visited.append(start_node)
    for neighbor in graph.get(start_node, []):
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
    return visited

if __name__ == "__main__":
    start = "Depot"
    print("BFS:", " -> ".join(bfs(graph, start)))
    print("DFS:", " -> ".join(dfs(graph, start)))
```

14. Important Code Lines

- **visited = []**: stores already visited nodes.
- **queue = [start_node]**: initializes the BFS queue.
- **queue.pop(0)**: removes the first item, giving FIFO behavior.
- **graph.get(current, [])**: safely obtains neighbors; missing keys give an empty list.
- **if neighbor not in visited**: avoids repeated processing.
- **dfs(...)**: recursively explores the next branch.
- **if __name__ == '__main__':**: runs the demo when the file is executed directly.

15. BFS vs DFS

Point	BFS	DFS
Meaning	Breadth-First Search	Depth-First Search
Exploration	Level by level	Deep path first
Data structure	Queue	Stack / recursion
This graph's result	Depot → Ward 1 → Ward 2 → Ward 3 → Ward 4	Depot → Ward 1 → Ward 3 → Ward 4 → Ward 2
Time (adjacency list)	$O(V + E)$	$O(V + E)$

16. Complexity Summary

- Binary Tree traversal: $O(n)$, because each node is visited once.
- BST average search/insert: $O(\log n)$ when balanced.
- BST worst-case search/insert: $O(n)$ when skewed.
- BFS with adjacency list: $O(V + E)$.

- DFS with adjacency list: $O(V + E)$.

17. Viva Questions & Answers

Q: What is a binary tree?

A: A tree where every node has at most two children.

Q: What is a BST?

A: A binary tree that also follows the left-smaller and right-larger ordering rule.

Q: What is the difference between binary tree and BST?

A: A binary tree has no required ordering; a BST has an ordering rule.

Q: What is a leaf node?

A: A node with no children.

Q: What is inorder traversal?

A: Left $\rightarrow$ Root $\rightarrow$ Right.

Q: Why is BST inorder sorted?

A: Because BST ordering places smaller keys on the left and larger keys on the right.

Q: What is BFS?

A: A graph traversal that explores level by level using a queue.

Q: What is DFS?

A: A graph traversal that explores deeply before backtracking, using a stack or recursion.

Q: Why do BFS and DFS use visited?

A: To prevent repeated visits and infinite loops in cyclic graphs.

Q: What does FIFO mean?

A: First In, First Out.

Q: What does LIFO mean?

A: Last In, First Out.

Q: What is an adjacency list?

A: A representation where each graph node stores its neighboring nodes.